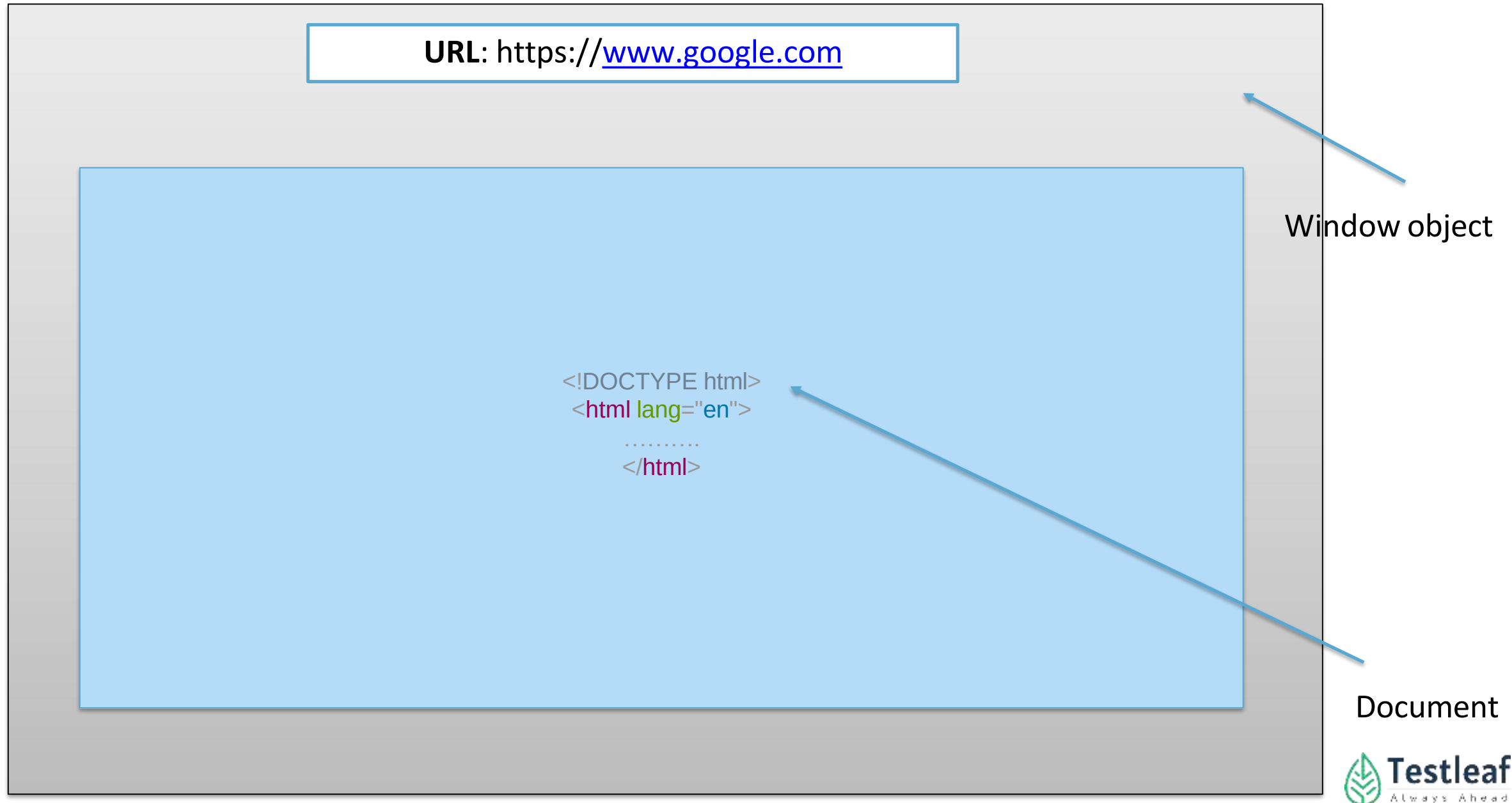
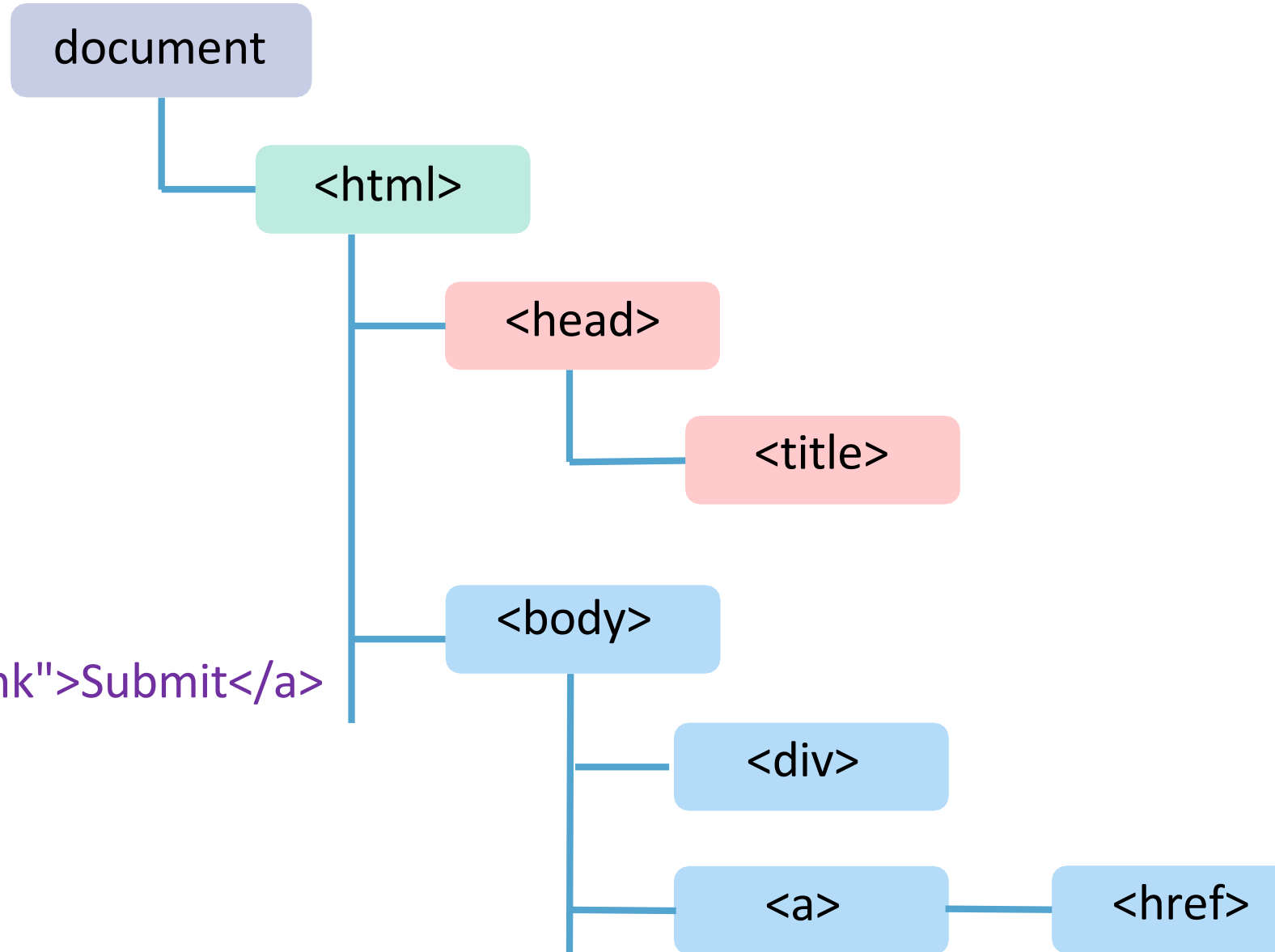


# DOM - Document object model



# DOM & WebElement

```
<!DOCTYPE html>
<html>
  <head>
    <title> DOM Structure</title>
  </head>
  <body>
    <div id="container">
      <h1>Welcome to Testleaf</h1>
      <p>This is a paragraph.</p>
      <a href="https://test.com" id="link">Submit</a>
    </div>
  </body>
</html>
```



WebElement

# Three core components of a webpage

- **HTML (HyperText Markup Language)**

Defines the structure and content of a webpage.  
Example: Headers, paragraphs, images, and forms.

- **CSS (Cascading Style Sheets)**

Controls the visual styling of the webpage.  
Example: Fonts, colors, layouts, and animations.

- **JavaScript**

Makes the webpage dynamic and interactive.  
Example: Form validation, animations, and real-time updates.

# Locators & Selectors

```
else{  
  (method) Page.locator(selector: string, options?: {  
    has?: Locator | undefined;  
    hasText?: string | RegExp | undefined;  
  } | undefined): Locator  
}
```

The method returns an element locator that can be used to perform actions on this page / frame. Locator is resolved to the element immediately before performing an action, so a series of actions on the same locator can in fact be performed on different DOM elements. That would happen if the DOM structure between those actions has changed.

[Learn more about locators.](#)

@param selector — A selector to use when resolving DOM element. See [working with selectors](#) for more details

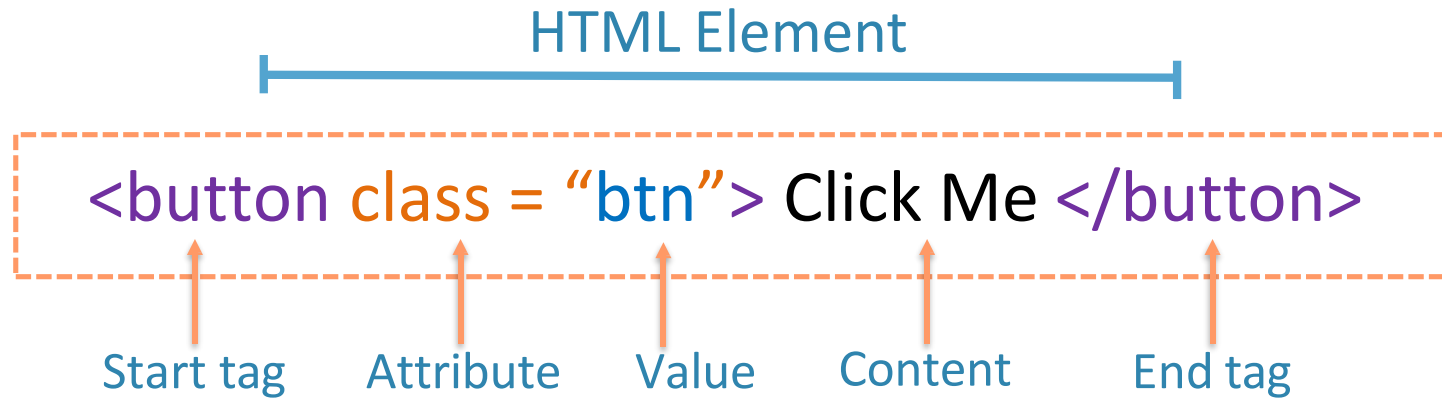
```
test('Locators'
```

```
  ⚡ await page.locator('#user-name')  
});
```

# Agenda – Part 3

- Locators
- Selectors
- Diving into Selector Strategies
- Which strategy to prefer?
- Industry best practices

# Let's understand structure of an element

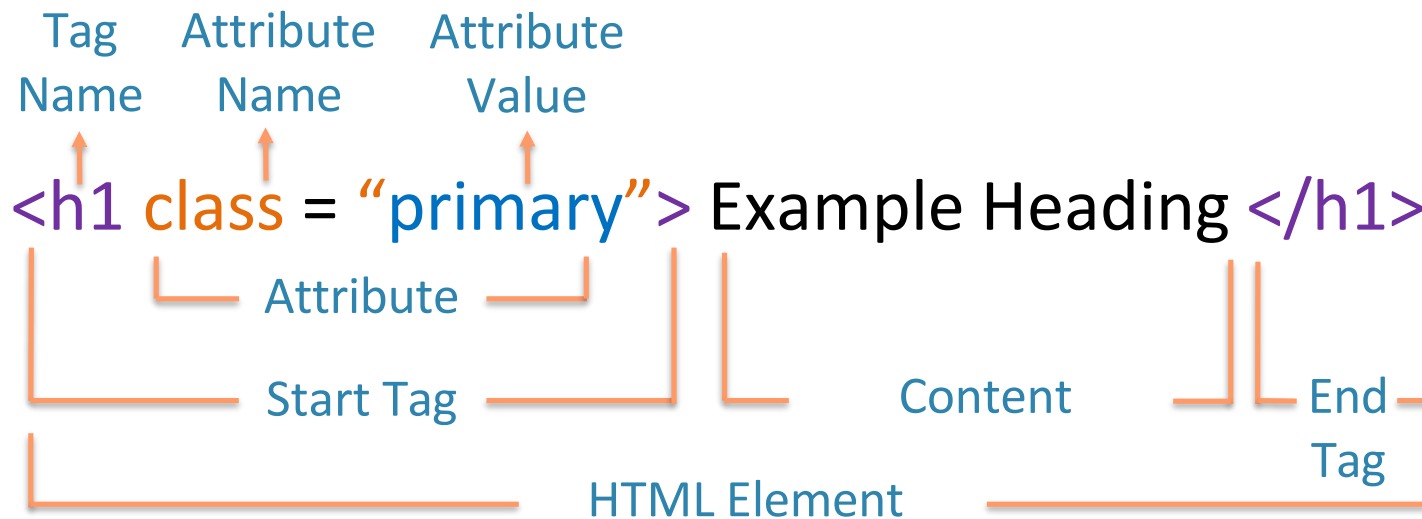


● Tags

● Attribute Name

● Attribute Value

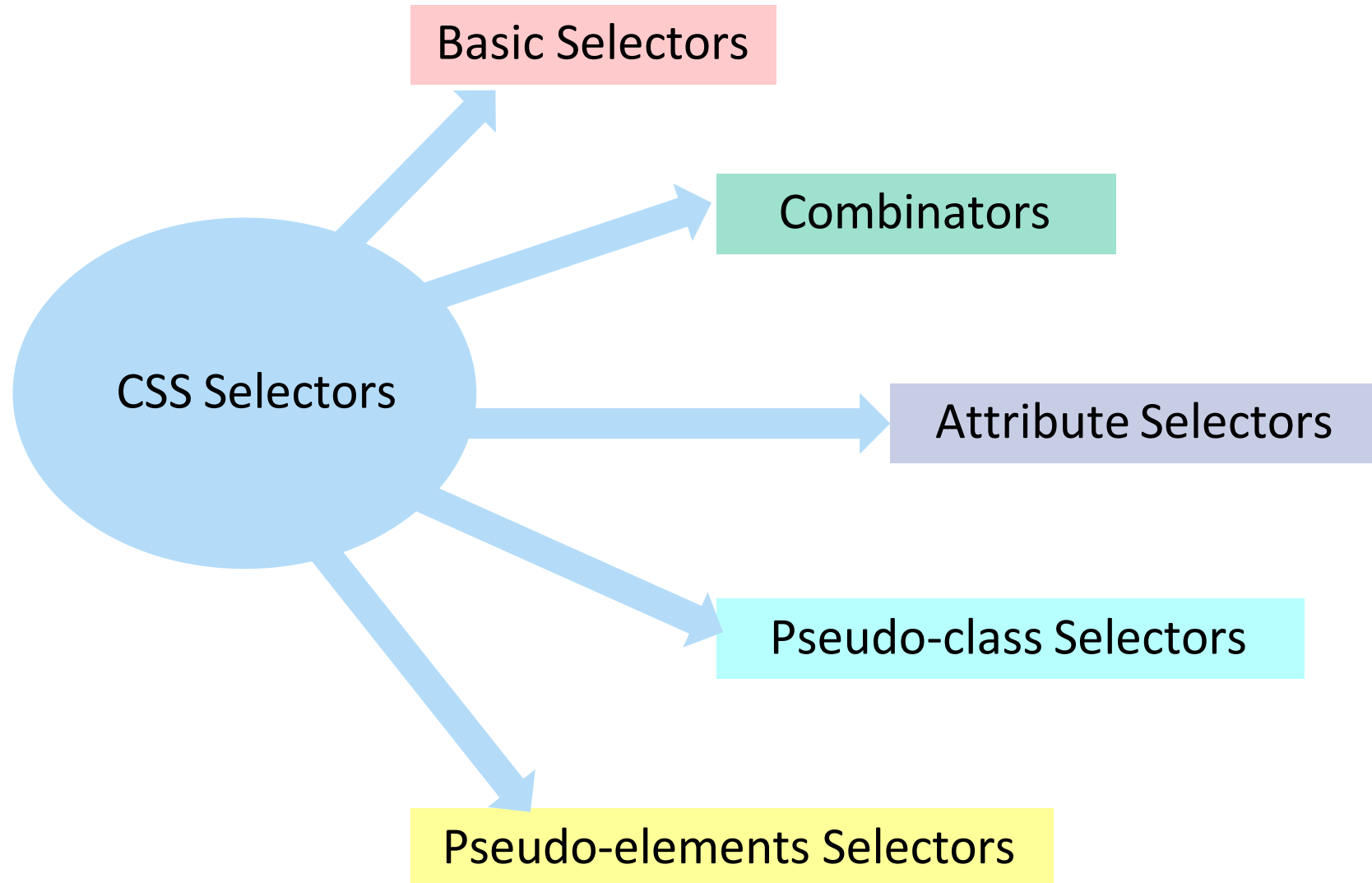
● Visible Text



# Diving into Selector Strategies

- CSS Selectors
- XPath Locators
- Playwright recommended selectors

# CSS Selectors



# CSS most common used strategies

- Find an element by tag name

*Syntax: tagName[attribute=value]*

- Find an element by ID

*Syntax: tagName[id=value]*

OR

*#idvalue*

Hash “#” to  
denote id

- Find an element by class

*Syntax: tagName[class=value]*

OR

*.classname*

Dot “.” to denote  
class

# CSS selectors for different situations

## Tag Name Selector

This selects the first <input> element on the page using the tag name input. In CSS, selecting by tag name means it will target all elements of that type unless further specified.

tagname <input>

*Syntax:* `await page.locator('input').nth(0).fill('crmsfa');`

## ID - #idValue

To locate a web element by its ID

`<input class="inputLogin" type="text" id="username" name="USERNAME" size="50">`

*Syntax:* `await page.locator("#username").fill('democr');`

# CSS selectors for different situations

## ● Class - .classValue

To locate a web element by its class

```
<input class="inputLogin" type="text" id="username" name="USERNAME" size="50">
```

*Syntax:* `await page.locator(".inputLogin").fill("demosalesmanager");`

## ● Attribute selector - [attributeName = 'attributeValue']

To locate by its HTML tag

```
<input class="inputLogin" type="text" id="username" name="USERNAME" size="50">
```

*Syntax:*

`await page.locator("input[id=username]").fill('democsr');`

# CSS selectors for different situations

## ● Descendant Combinator (ancestor descendant)

Selects all elements that are *descendants* (not just direct children) of a specified ancestor.

*Example :*

selects all `<input>` elements inside a `<form>`, regardless of nesting level.

*Syntax :* `form input`

## ● Child Combinator (parent>child)

Selects elements that are direct children of a specified parent.

*Example :*

selects `<input>` elements that are directly inside a `<p>` tag, not deeper nested inputs

*Syntax :*

`p > input.`

# CSS selectors for different situations

- **Adjacent Sibling Combinator (element + adjacent)**

Selects an element that is the immediate next sibling of another element.

*Example :*

*selects the first <input> immediately following a <label>*

*Syntax :* label + input or meta+link

- **General Sibling Combinator (element ~ sibling)**

Selects all siblings that come after a specified element, not necessarily immediately.

*Example :*

*selects all <p> elements that follow an <h2> in the same parent, even if other elements are in between.*

*Syntax :* meta~link

# Industry Best Practices

- Writing locator which are unique
- Writing locator which are readable and can provide some context
- Using text based locator
- Using a unique attribute dedicated for testing like data test id etc.,

# Playwright *config* file walkthrough

- *testDir* Contains dir path where all test cases resides
- *fullyParallel* Decides if to run all files in parallel or all. Files and test cases inside them in parallel
- *Test timeout* Max time allowed for a test case to run
- *retries* Number of time a test case to retry on failure
- *workers* Number of workers to split the test cases on
- *trace* On what conditions, we should keep the trace
- *screenshots* On what conditions, we should keep the screenshot
- *video* On what conditions, we should keep the video
- *Expect timeout* Max time allowed for assertions we are doing using expect

# Playwright *config* file walkthrough

## Key Differences in CLI Execution

| Feature              | Using <code>page</code> Fixture                        | Manually Creating Browser/Context/Page    |
|----------------------|--------------------------------------------------------|-------------------------------------------|
| Browser Creation     | Managed by Playwright                                  | Needs to be manually launched             |
| Context per test     | New context for each test                              | Context persists across tests             |
| Parallel Execution   | Default parallel tests                                 | Must manage manually                      |
| CLI Options Applied? | ✅ Yes, CLI overrides <code>playwright.config.ts</code> | ❌ No, must be coded manually              |
| Best For             | Simple tests, CLI overrides, parallel execution        | Full browser control, persistent sessions |

# Auto Wait

*Playwright when performing any relevant action first ensures that*

- All applied checks for that actions are passed
- Within a set timeout which can be overridden by
- If the criteria is not full filled , it will raise ***TimeoutError***

`page.click()` ensures

- element is attached
- element is visible
- element is stable
- element is receiving events
- element is enabled